

Oblig1 – IN2010

Oppgave 2a)

```
Public void push_back(int antallBak)  
new node  
IF størrelse == 0  
    node = stok, midle, slutt  
  
    node sin forrige blir slutt  
IF slutt ikke er null  
    slutt sin neste blir node  
  
slutt blir til node  
OK størrelse
```

```
Public void push_front(int antallFørste))  
    new node
```

```
IF størrelse == 0
```

```
    node = start, midt, slutt
```

```
    node.neste er start
```

```
IF start ikke er null
```

```
    start sin forrige = node
```

```
start = node
```

```
OK størrelse til køen
```

```
Public void push-middle(int aneMidt)  
new node
```

```
IF størrelse == 0
```

```
node = start, midt, slutt
```

```
ELSE
```

```
new midtNode er start
```

```
FOR  $1 \rightarrow (størrelse + 1) / 2$ 
```

```
midtNode er midtNode sin næste
```

```
midt er midtNode
```

```
node sin forrige er midt
```

```
IF midt ikke er null
```

```
node sin næste er midt
```

```
midt sin næste er node
```

```
midt er node
```

```
OK størrelse
```

```
Public int get (int x)
    new node er score
    FOR 0 → y
        node er node sin neste
    RETURN node sin data
```

Oppgave 2c)

Hvis N ikke er begrenset blir verste-tilfelle kjøretidsanalysen slik:

push_back --> $O(1)$

push_front --> $O(1)$

push_middle --> $O(n)$

get --> $O(n)$

Hvis N er begrenset blir verste-tilfelle kjøretidsanalysen slik:

push_back --> $O(1)$

push_front --> $O(1)$

push_middle --> $O(1)$

get --> $O(1)$

Oppgave 2d)

Grunnen til at det er viktig at fjerner begrensningen på N fra forrige deloppgave kommer av at tiden vil bli en konstant. Konstanter blir utelatt i en verste tilfelle kjøretidsanalyse, siden det påvirker økningen til funksjonen en konstant mengde. På grunn av dette vil det ikke være hensiktsmessig å foreta en slik kjøretidsanalyse på en begrenset konstant N.

Oppgave 3a)

```
WHILE kettegrensen ikke er -1
    sett inn enll av linjesykkel som key-
    og seot av linjesykkel som value

FOR 0 < nognMod.Sin Scoerselge
    print kettegrensen til den ikke lenger er der
```

Oppgave 4a)

```
Public static void lestSortertArray (array, start, slutt)
    IF start er mindre enn slutt
        midtElement ← (start + slutt) / 2
        PRINT array sitt midtElement
        kall metoden med (array, midtElement+1, slutt)
        kall metoden med (array, start, midtElement-1)
    ELSE
        RETURN
```

Oppgave 4b)

```
Public static void lessortHeap (pk, start, slutt)
    størrelse ← pk sin size()
    midtElement ← (start + slutt) / 2
    IF størrelse er større enn 1
        lager en ny priorityQueue nyPk
        teller ← 0
        WHILE teller er mindre enn (størrelse / 2)
            legger til element i nyPk og tar ut fra pk
            teller++

        PRINT elementet som ble pottet
        kall metoden med (pk, start, midtElement - 1)
        kall metoden med (nyPk, midtElement + 1, slutt)
    ELSE
        RETURN
```